

Relatório de Laboratório

Laboratório de Experimentação de Software — Modelo/Template de Relatório

Relatório Final do Laboratório 01 da disciplina Laboratório de Experimentação de Software (PUC Minas). Este documento descreve a mineração dos 1.000 repositórios mais estrelados do GitHub, as hipóteses informais do grupo, a metodologia de coleta via GraphQL, a análise e visualização gráfica das sete RQs (incluindo o bônus RQ07) e a configuração do processo Kanban no GitHub Projects.

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 — Características de repositórios populares + Setup do Kanban
Grupo (trio)	Gabriel Ferreira Amaral (Druitti) · Gabriel Santiago (gabsant07) · Gabriel Tavares (Tavaresds1)
Link do repositório / GitHub Projects	Repo: https://github.com/Druitti/Laboratorio-de-Desenvolvimento-de-Software Project: https://github.com/users/Druitti/projects/5
Data de entrega	25/08/2026

1. Introdução

Perguntas que esta seção deve responder ao leitor

- Qual problema está sendo investigado, e por que ele importa (para a engenharia de software, para o mercado, ou para o próprio grupo)?
- Quais são as Questões de Pesquisa do enunciado (numeradas RQ1, RQ2, ...)?
- Quais as hipóteses informais do grupo para cada RQ, antes de olhar os dados (quando aplicável ao laboratório)?
- Quais RQs, métricas ou variáveis o grupo está propondo além do enunciado (resumo de 1 linha cada — os 30% de inovação)?

Repositórios open-source populares concentram boa parte da atenção da indústria e da academia, mas nem sempre há evidência sistemática sobre o perfil desses sistemas (idade, contribuição externa, releases, atualização, linguagem e gestão de issues). Este laboratório investiga essas características nos 1.000 repositórios com maior número de estrelas no GitHub e, em paralelo, institui o quadro Kanban (GitHub Projects v2) que acompanhará o grupo no semestre.

Questões de Pesquisa do enunciado:

RQ01. Sistemas populares são maduros/antigos? (idade a partir de `createdAt`)

RQ02. Sistemas populares recebem muita contribuição externa? (PRs merged)

RQ03. Sistemas populares lançam releases com frequência? (total de releases)

RQ04. Sistemas populares são atualizados com frequência? (tempo desde `pushedAt`)

RQ05. Sistemas populares são escritos nas linguagens mais populares? (`primaryLanguage` vs TIOBE Index)

RQ06. Sistemas populares possuem alto percentual de issues fechadas? (`closed/total`)

RQ07 (bônus). Linguagens TIOBE associam-se a mais PRs, releases e atualizações mais recentes?

Hipóteses informais (antes da análise completa — Issue #9):

RQ01 — Sim: mediana de idade elevada (vários anos), pois popularidade acumula-se no tempo.

RQ02 — Em parte: mediana de PRs merged relevante, com forte assimetria (poucos repos concentram milhares de PRs).

RQ03 — Não necessariamente: mediana de releases baixa/próxima de zero por causa de repos de curadoria/docs.

RQ04 — Sim: tempo desde o último push tipicamente curto (dias/poucas semanas).

RQ05 — Parcialmente: parcela relevante em linguagens TIOBE, mas também Markdown/null e linguagens fora do índice (ex.: TypeScript).

RQ06 — Sim: razão mediana `closed/total` alta (~70–80% ou mais).

RQ07 — Mista: TIOBE com mais PRs e updates mais recentes; releases com diferença menor.

Inovações (30%): (i) validação rápida de amostra (5–10 repos) por bloco de métricas antes da coleta completa; (ii) paginação em lotes menores com `retry/backoff` para mitigar HTTP 502 na API GraphQL; (iii) snapshots CSV do GitHub Projects para reconstituição histórica do board (base dos Labs 04/05).

2. Contexto

Este é o Lab01, ponto de partida do semestre: combina mineração de software (1.000 repositórios mais populares do GitHub) com a implantação do processo Kanban do próprio grupo. Os dados e o board gerados aqui alimentam laboratórios posteriores (em especial snapshots para Labs 04 e 05).

Objeto de estudo: repositórios retornados pela busca GraphQL ``stars:>1 sort:stars-desc``, limitados aos 1.000 primeiros, com métricas alinhadas às RQs 01–06 (e campos necessários ao bônus RQ07).

Referência conceitual para “linguagens mais populares” (RQ05/RQ07): TIOBE Index (<https://www.tiobe.com/tiobe-index/>), mantida como fonte única em todo o Lab01, conforme documentado no README e em docs/hipoteses-informais.md.

3. Metodologia

3.1 Principais Desafios

Principais desafios enfrentados:

- Complexidade/instabilidade da API GraphQL do GitHub ao solicitar connections (PRs, issues, releases) para dezenas de repositórios na mesma página — ocorrência recorrente de HTTP 502, mitigada com páginas menores e retries com backoff.
- Paginação até 1.000 itens respeitando o limite de 100 nós por página da API Search.
- Ausência de histórico consultável de mudança de Status no GitHub Projects via API, exigindo exportações periódicas (snapshots CSV) do board.
- Valores ausentes: `primaryLanguage` nulo (listas/docs) e repositórios com zero issues (razão RQ06 indefinida).
- Restrição do enunciado: proibição de bibliotecas de terceiros que encapsulem a API do GitHub — a query GraphQL e o cliente HTTP são próprios do grupo (Python + requests).

3.2 Tomadas de Decisão

Tomadas de decisão:

- Linguagem/script: Python 3 com requests e query GraphQL própria (sem SDK GitHub).
- Amostra: top 1.000 por estrelas via search GraphQL; sem exclusão a priori por linguagem.
- RQ04 operacionalizada como dias desde `pushedAt` (atividade de código); `updatedAt` fica como apoio.
- RQ05: classificação TIOBE vs. não-TIOBE a partir de `primaryLanguage.name` (TypeScript, Markdown etc. contam como fora do conjunto TIOBE adotado).
- WIP da coluna Doing = 2: trio; evita excesso de paralelismo e força Review/Done frequentes

(docs/processo-kanban.md).

- Commits devem referenciar #Issue para vinculação no board (correção centrada no Projects).

3.3 Etapas

Etapas (sprints) e responsáveis (refletindo Assignees das Issues):

Lab01S01 — Coleta GraphQL (100 repos) + setup Kanban

- #2 Setup Projects/WIP/labels — Gabriel Ferreira (Druitti) — Done
- #1 Query base + auth + fetch — Druitti — Done
- #3 RQ01+RQ02 + validação amostra — Druitti — Done
- #4 RQ03+RQ04 + validação — Druitti — Done
- #5 RQ05+RQ06 + TIOBE — Druitti — Done
- #6 Integração/smoke 100 — Druitti — Done

Lab01S02 — Paginação 1000 + CSV + hipóteses + snapshot

- #7 Paginação 1000 + export CSV — Gabriel Santiago (gabsant07) — Done (código + dados gerados)
- #8 Snapshot GraphQL do Project → CSV — gabsant07 — em aberto no board (código presente: src/collect_repos_snapshot.py)
- #9 Hipóteses informais — Druitti — Done

Lab01S03 — Análise/visualização

- #10 Análise/visualização RQs + bônus — Gabriel Tavares (Tavaresds1) e gabsant07 — Done

Relatório Final

- #11 Relatório final + print do board — Druitti — este documento

Configuração do processo (Kanban):

- Colunas Status: Backlog → To Do → Doing → Review → Done
- WIP Doing = 2 (justificativa na seção 3.2 e em docs/processo-kanban.md)
- Cartões = Issues reais com Assignee; labels por integrante

[INSERIR AQUI O PRINT DO BOARD] — Anexo obrigatório: captura de tela do GitHub Projects (<https://github.com/users/Druitti/projects/5>) ao final do Lab01, mostrando o fluxo completo e a política de WIP. Salvar também em relatorio/anexos/board-lab01.png se desejar versionar a imagem.

Sugestão de anexo: print do quadro Kanban (GitHub Projects) mencionado acima. Enquanto a imagem não for colada neste .docx, use o link do Project e/ou o arquivo em relatorio/anexos/.

3.4 Ferramentas

Ferramentas utilizadas:

- GitHub GraphQL API (<https://api.github.com/graphql>) — mineração e snapshot do Project
- Python 3 + requests (+ python-dotenv opcional) — scripts próprios do grupo (src/collect_repos.py, src/validate_sample.py, src/collect_repos_snapshot.py)
- Git / GitHub Issues + GitHub Projects (v2) — processo
- Python 3 + matplotlib + seaborn — geração dos gráficos das RQs 01–07 (scripts/gerar_graficos_rqs.py)

Links:

- Repositório: <https://github.com/Druitti/Laboratorio-de-Desenvolvimento-de-Software>
- GitHub Projects: <https://github.com/users/Druitti/projects/5>

3.5 Tabela de Métricas

ORIENTAÇÃO: Construa uma tabela relacionando cada Questão de Pesquisa à métrica correspondente, sua definição operacional exata (a fórmula ou regra de cálculo — não basta o nome) e a ferramenta/fonte usada para coletá-la. Isso é o que garante que o laboratório seja reproduzível por outro grupo. A primeira linha abaixo é um exemplo ilustrativo (baseado no Lab01); substitua pelas RQs e métricas do seu laboratório.

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do repositório	Data da coleta – createdAt	Dias	Script GraphQL próprio (API GitHub)
RQ02	PRs aceitas	pullRequests(states:MERGED).totalCount	Contagem	Script GraphQL próprio
RQ03	Releases	releases.totalCount	Contagem	Script GraphQL próprio
RQ04	Tempo até última atualização	Data da coleta – pushedAt	Dias	Script GraphQL próprio
RQ05	Linguagem primária	primaryLanguage.name classificada vs TIOBE Index	Categoria	GraphQL + TIOBE Index
RQ06	% issues fechadas	closedIssues.totalCount / issues.totalCount (se total>0)	Proporção	Script GraphQL próprio
RQ07 (bônus)	PRs/releases/atualização por linguagem	Segmentação das métricas RQ02–04 por linguagem / TIOBE	Múltipla	scripts/gerar_graficos_rqs.py

3.6 Inovações Propostas pelo Grupo (30% da nota)

Inovações propostas pelo grupo (30%):

- 1) Pipeline de validação de amostra (validate_sample.py) por bloco de RQs antes da coleta em escala — reduz risco de métrica ausente/errada.
- 2) Estratégia anti-502 (page size reduzido + retries) na coleta GraphQL dos top repositórios.
- 3) Exportação de snapshots do GitHub Projects em CSV (collect_repos_snapshot.py), criando série

temporal do Status das Issues para Labs 04/05.

Os resultados numéricos dessas inovações aparecem na robustez da coleta (1.000 repos obtidos) e na rastreabilidade do processo; gráficos comparativos detalhados constam na seção 4.2.

4. Resultados

4.1 Coleta de Dados

ORIENTAÇÃO: *Relate o volume final de dados efetivamente coletado e analisado — não apenas o volume-alvo do enunciado. Informe: quantos itens restaram após os filtros de qualidade (ex.: dos 1.000 repositórios buscados, quantos tinham dados completos; dos repositórios candidatos, quantos de fato usavam GitHub Actions — Lab03); o período coberto pela coleta; quantos trials/execuções foram concluídos dentro do tempo (Lab02); quantos snapshots do Kanban estão disponíveis e desde quando (Lab04/Lab05); outliers ou dados ausentes identificados, e como foram tratados (removidos, mantidos e discutidos à parte, etc.).*

Coleta de dados (estado atual):

- Alvo: 1.000 repositórios; obtido: 1.000 linhas em data/top_1000_repos.csv (e JSON correspondente).
- Período de referência da coleta local disponível: agosto/2026.
- Ausências observadas na amostra: primaryLanguage nulo em 87 repos; 43 repos com zero issues (razão RQ06 não definida).
- Tratamento: ausências reportadas; razões RQ06 calculadas apenas quando issues > 0; linguagem nula tratada como categoria própria / fora do TIOBE.

Resumo descritivo (medianas na amostra de 1.000; ver gráficos na seção 4.2):

- RQ01 — mediana da idade ≈ 2.825 dias ($\sim 7,7$ anos)
- RQ02 — mediana de PRs merged ≈ 768 (média ≈ 4.243 — forte assimetria)
- RQ03 — mediana de releases ≈ 40 ; cerca de 28% com zero releases
- RQ04 — mediana de dias desde pushedAt $\approx 1,2$ (repositórios extremamente ativos no topo do ranking de estrelas)
- RQ05 — entre repos com linguagem: $\sim 67,9\%$ em linguagens do conjunto TIOBE adotado; tops: Python, TypeScript, JavaScript, Go, Rust
- RQ06 — mediana closed/total $\approx 0,875$ (87,5%)
- RQ07 (bônus) — TIOBE vs. Não TIOBE: PRs merged (mediana 865 vs. 1.000), releases (mediana 51 vs. 51), dias desde push (mediana 0,6 vs. 0,3) — diferenças pequenas entre os dois grupos

Snapshots do Project: script disponível (src/collect_repos_snapshot.py); a Issue #8 permanece aberta no board até a exportação CSV de fechamento ser confirmada pelos responsáveis.

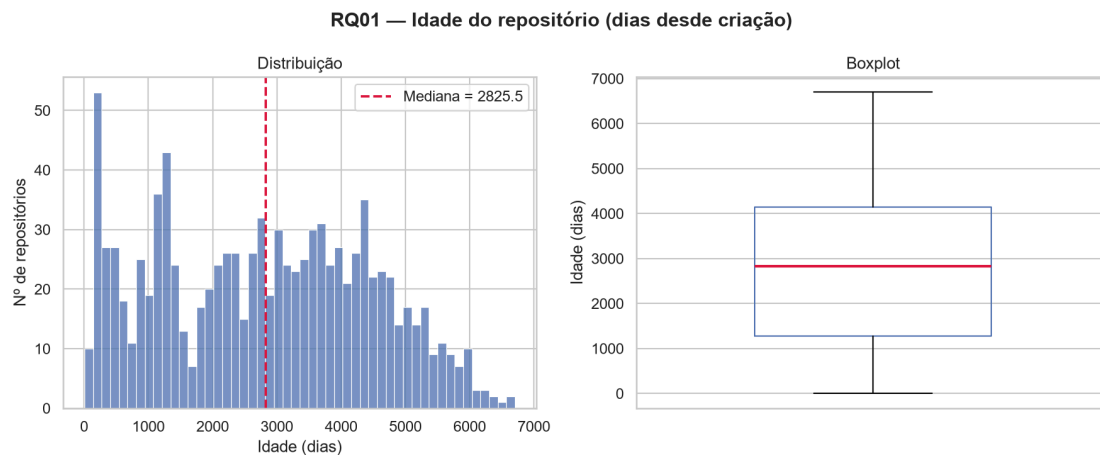
4.2 Visualização Gráfica

Tipo de pergunta / dado	Gráfico recomendado
-------------------------	---------------------

Comparar uma métrica entre categorias (ex.: linguagem, benchmark DORA)	Barras (ranking) — ordenadas por valor, não alfabeticamente
Comparar dois tratamentos no mesmo grupo (ex.: com IA vs. sem IA)	Boxplot pareado ou gráfico de pontos conectados (before/after)
Distribuição de uma métrica numérica (ex.: idade dos repositórios)	Histograma ou boxplot
Relação entre duas métricas numéricas (ex.: RQ07 do Lab01, RQ05 do Lab03)	Gráfico de dispersão (scatter plot)
Evolução ao longo do tempo (ex.: cycle time por sprint)	Linha, com um ponto por sprint/snapshot
Composição/fluxo do Kanban ao longo do tempo (Cumulative Flow Diagram)	Área empilhada (uma camada por coluna do board)
Proporção de categorias (ex.: % de issues fechadas)	Barra única 100% ou barras simples — evite pizza com muitas fatias

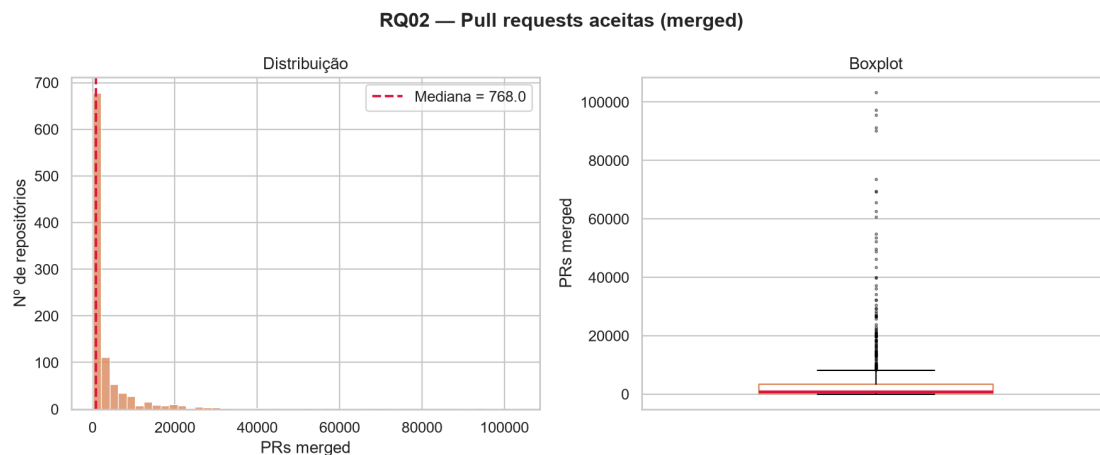
A seguir, um gráfico por RQ (mais o comparativo do bônus RQ07), gerados por `scripts/gerar_graficos_rqs.py` a partir de `data/top_1000_repos.csv`, com a mediana destacada em cada distribuição.

RQ01 — Sistemas populares são maduros/antigos?



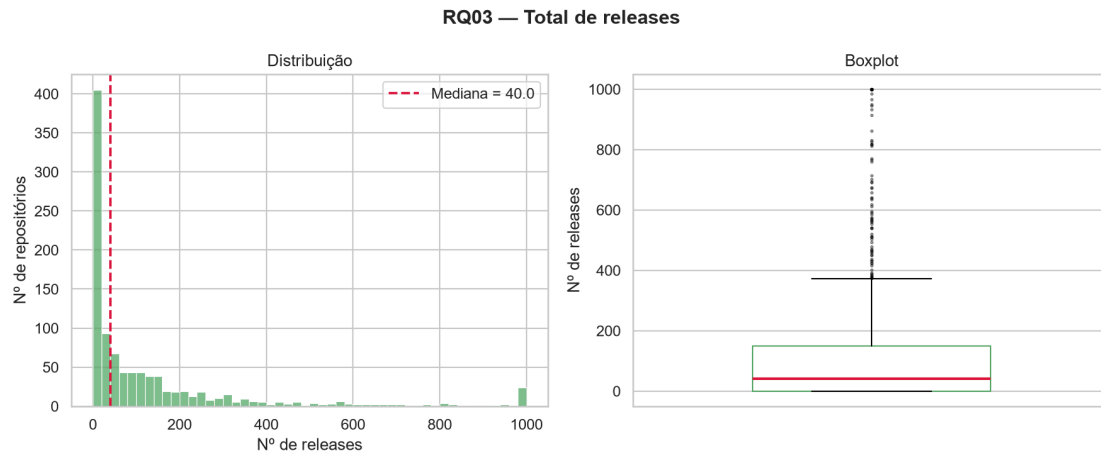
Mediana: 2.825 dias (~7,7 anos).

RQ02 — Sistemas populares recebem muita contribuição externa?



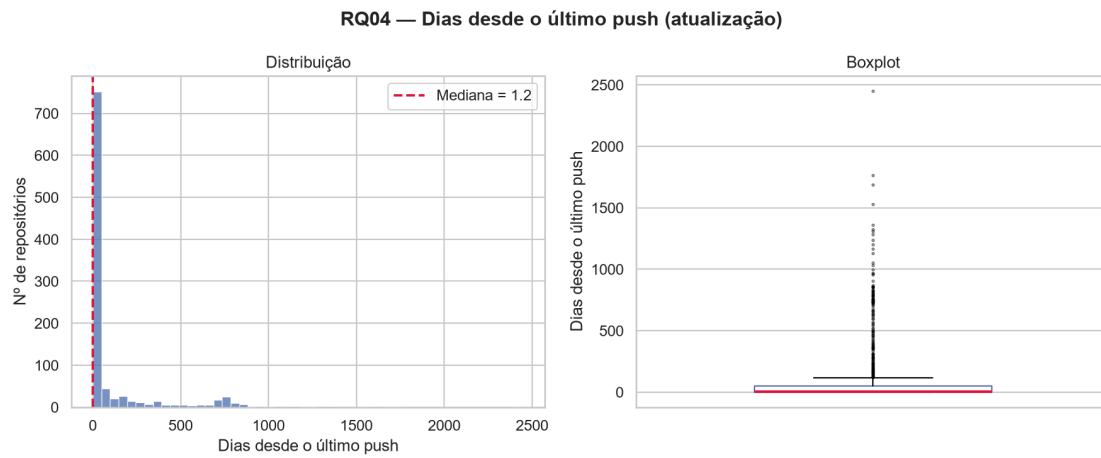
Mediana: 768 PRs merged.

RQ03 — Sistemas populares lançam releases com frequência?



Mediana: 40 releases; 28% dos repositórios sem nenhuma release.

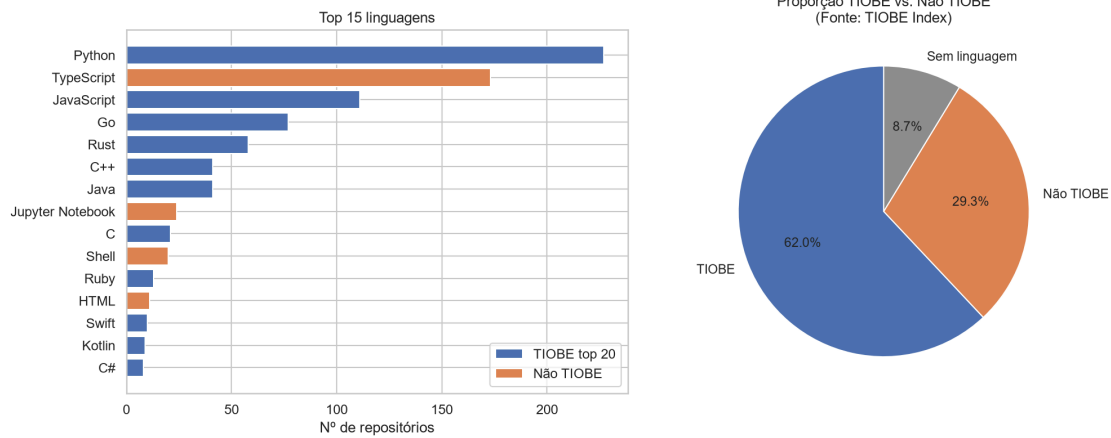
RQ04 — Sistemas populares são atualizados com frequência?



Mediana: 1,2 dia desde o último push.

RQ05 — Sistemas populares são escritos nas linguagens mais populares?

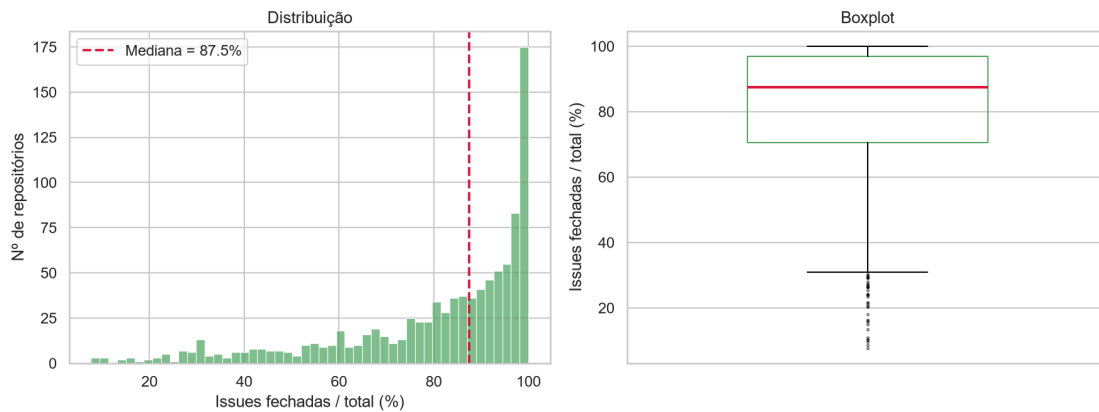
RQ05 — Linguagem primária dos repositórios populares



62,0% em linguagens TIOBE top 20; 29,3% fora do TIOBE; 8,7% sem linguagem primária.

RQ06 — Sistemas populares possuem um alto percentual de issues fechadas?

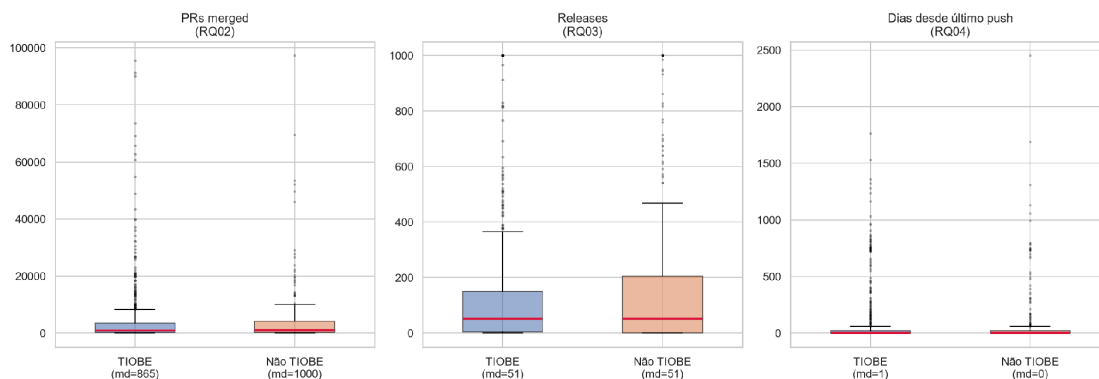
RQ06 — Percentual de issues fechadas



Mediana: 87,5% de issues fechadas.

RQ07 (bônus) — Linguagens TIOBE recebem mais contribuição, lançam mais releases e são atualizadas com mais frequência?

RQ07 (bônus) — Comparação TIOBE vs. Não TIOBE



Medianas TIOBE vs. Não TIOBE: PRs merged 865 vs. 1.000; releases 51 vs. 51; dias desde push 0,6 vs. 0,3 — diferenças pequenas entre os grupos.

4.3 Discussão

ORIENTAÇÃO: Para cada RQ (do enunciado e das RQs de inovação do grupo), compare explicitamente a hipótese informal levantada na Introdução com o resultado efetivamente obtido — hipótese confirmada, refutada, ou parcialmente confirmada, e por quê. Quando houver teste estatístico (ex.: Wilcoxon no Lab02), reporte o valor obtido e interprete o que ele significa em linguagem acessível, não apenas o número bruto. Discuta as ameaças à validade específicas do laboratório (ex.: efeito de aprendizado entre katas e risco de memorização pela IA — Lab02; diferença de dificuldade entre laboratórios distintos confundindo a tendência de cycle time — Lab05; lacunas nos snapshots do Kanban — Lab05). Finalize relacionando o que as inovações do grupo (seção 3.6) acrescentaram: elas confirmaram, contradisseram ou aprofundaram o que os 70% do enunciado já mostravam?

Discussão hipótese × evidência descritiva (medianas), com apoio das visualizações da seção 4.2.

RQ01 — Hipótese confirmada: mediana ~7,7 anos indica maturidade elevada.

RQ02 — Hipótese parcialmente confirmada: mediana 768 PRs merged é alta, mas a média muito superior evidencia assimetria (cauda longa).

RQ03 — Hipótese parcialmente confirmada: há mediana > 0 (40), porém 28% sem releases — compatível com mistura de software versionado e curadoria.

RQ04 — Hipótese confirmada: mediana de apenas ~1,2 dia desde o último push indica atualização praticamente diária nos repositórios mais populares.

RQ05 — Hipótese parcialmente confirmada: maioria (entre os com linguagem) no TIOBE, mas TypeScript/Markdown/null mostram que popularidade ≠ apenas top TIOBE.

RQ06 — Hipótese confirmada: mediana 87,5% de issues fechadas.

RQ07 — Hipótese não confirmada: as medianas de PRs merged, releases e dias desde push são muito próximas entre repositórios TIOBE e Não TIOBE (865 vs. 1.000 PRs; 51 vs. 51 releases; 0,6 vs. 0,3 dias), indicando que a linguagem primária não é um fator discriminante relevante para essas métricas nesta amostra.

Ameaça à validade: Search API ordena por estrelas e pode omitir/alterar ranking ao longo do tempo; primaryLanguage nulo e repos sem issues afetam RQ05/RQ06; PRs merged medem contribuição aceita, não necessariamente “externa” no sentido estrito (contribuições de maintainers também entram); a comparação da RQ07 é feita por medianas descritivas, sem teste estatístico formal.

5. Conclusão

ORIENTAÇÃO: Sintetize, em poucos parágrafos, as respostas a todas as RQs (enunciado + inovação do grupo), sem repetir números já discutidos em detalhe — o objetivo aqui é a mensagem final, não os dados brutos. Aponte as principais limitações do estudo (tamanho de amostra, ameaças à validade não mitigadas, período de coleta). Quando o enunciado pedir explicitamente uma postura de consultoria (caso do Lab05, que pede recomendações de melhoria de processo “como se o grupo fosse consultoria para um time real”), inclua recomendações objetivas e acionáveis, não genéricas. Encerre indicando o que o grupo faria diferente

com mais tempo ou recursos, e quais das inovações propostas (30%) valeriam a pena expandir em um trabalho futuro.

O Lab01 permitiu caracterizar, de forma exploratória, o perfil dos 1.000 repositórios mais estrelados do GitHub e institucionalizar o Kanban do grupo (colunas Backlog→Done, WIP=2, Issues com Assignee e commits referenciando #N).

Em síntese descritiva: os sistemas populares tendem a ser maduros e atualizados com frequência, com razão alta de issues fechadas e contribuição via PRs assimétrica; a relação com linguagens TIOBE é parcial na composição (RQ05), mas não se traduz em diferenças relevantes de contribuição, releases ou atualização entre os grupos TIOBE e Não TIOBE (RQ07); releases variam bastante.

Limitações: dependência da Search API; métricas proxy (ex.: PRs merged $\neq$ contribuição externa pura); a comparação da RQ07 usa apenas medianas descritivas, sem teste estatístico formal; print do board deve ser anexado neste documento.

Com mais tempo, o grupo aplicaria testes estatísticos formais (ex.: Mann-Whitney) para validar as diferenças da RQ07 e automatizaria snapshots a cada fechamento de sprint no CI.

5. Referências

- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.
-

Grupo Lab01 — repositório e board:

<https://github.com/Druitti/Laboratorio-de-Desenvolvimento-de-Software> ;

<https://github.com/users/Druitti/projects/5>

TIOBE Software BV. TIOBE Index. Disponível em: <https://www.tiobe.com/tiobe-index/>. Acesso em: 25 ago. 2026.

GitHub. GraphQL API documentation. Disponível em: <https://docs.github.com/en/graphql>. Acesso em: 25 ago. 2026.

BASIL, Victor R.; CALDIERA, Gianluigi; ROMBACH, H. Dieter. The Goal Question Metric Approach. 1994.